

드림핵 Assembly Language

학습
위계임
CTF
커뮤니티
챌린지
스토어
구독

Assembly Language

★ 10.0 (5)

Tier D Beginner Skill Path Beginners

시스템 해킹, 리버스 엔지니어링을 공부하기 전 학습하기 적절한 과정입니다. 기초적인 도구 사용법부터 컴퓨터 과학 이론, 자세한 실습 중심의 커리큘럼을 통해 어셈블리어 작성 역량을 갖추고, 우후 순차한 분야를 학습하는 데 필요한 능력을 체계적으로 얻을 수 있습니다.

학습목표

- Unit 구성
- 수강 후기 5

숙여요. Path 수강을 모두 완료하셨군요!

무기를 날기고 작은 선물도 받아주세요.

[수강을 작성해가 >](#)

이런 이유로 이 Path를 추천해요

어셈블리어는 CPU 명령어와 메모리 동작을 가장 낮은 수준에서 직접 제어할 수 있는 저수준 언어로, 시스템 해킹, 리버스 엔지니어링 등 다양한 보안 분야에서 핵심적인 역할을 합니다. 특히 디버거를 활용해 복잡한 프로그램을 분석할 때도 어셈블리 수준에서 코드 흐름과 레지스터 상태를 이해해야 동작을 정확하게 분석할 수 있습니다. 이 Path를 통해 어셈블리어의 기본 문법과 시스템 콜, 레지스터 관리 등 핵심 개념을 차근차근 익히고, 직접 어셈블리의 코드를 작성해보는 실습을 통해 저수준 프로그래밍에 대한 깊이 있는 이해를 얻을 수 있습니다.

이런 내용을 배워요

- 어셈블리어의 기본 개념과 문법
- NASM 어셈블러 설치 및 활용법
- 다양한 아키텍처 및 운영체제의 할 수 조율 규약
- 어셈블리어를 활용한 프로그램 실행

이런 분께 추천해요

- 어셈블리어를 처음 접하거나 개념을 체계적으로 정리하고 싶은 분
- 어셈블리 코딩에 나 자신 코드 작성 능력 향상 시킬 분

보유 중

Path 상세

- 🕒 약 15시간 30분
- 📁 12개의 강의
- 🔧 1개의 Lab
- 📄 2개의 문제
- 🏆 8개의 퀴즈
- 👤 Beginner, 매우 쉬운 난이도

100% 수강완료

총 22개 항목 완료

📅 강의 12 / 12 🔖 레이블

🗂 위계임 2 / 2

🎯 퀴즈 8 / 8

qwevjiqp
@jpxkfhghnawen.com

프로필 커스텀

내 VNM 크레딧 00

내 유료 콘텐츠 0

내 무료 콘텐츠 10

🔍 마이메시지

📊 계정 설정

🛠 로그아웃

Starter

컴퓨터 과학 기초

- 진법

임의의 숫자 혹은 문자를 사용하여 수를 표현하는 체계. 10진법, 2진법, 16진법, 8진법 등이 있다. 2진수는 0b라는 접두어를 붙이고, 16진수는 0x라는 접두어를 붙인다. 컴퓨터에서는 주로 2진법과 16진법을 사용한다.

- 비트와 바이트

컴퓨터의 데이터를 다루는 데 가장 기본이 되는 개념. 컴퓨터는 0과 1만으로 데이터를 표현하고 처리하는데, 이때 0과 1을 나타내는 컴퓨터에서 사용하는 데이터의 최소 단위를 1비트라고 한다. 8개의 비트로 구성된 더 큰 단위는 1바이트라고 하고 이는 메모리에 저장되는 최소 단위이다.

여러 개의 비트로 구성된 이진 데이터에서 가장 왼쪽에 있는 비트를 Most Significant Bit(MSB)라고 하고 가장 오른쪽에 있는 비트를 Least Significant Bit(LSB)라고 한다.

부호가 있는 데이터의 경우 MSB가 0이면 양수, 1이면 음수를 나타낸다.

- 바이트 오더링

바이트가 메모리에 저장되는 방식. 빅 엔디안은 큰 바이트부터 낮은 주소에 저장되고, 리틀 엔디안은 작은 바이트부터 낮은 주소에 저장한다.

문자열을 메모리에 저장할 때는 바이트 오더링을 고려하지 않는다. 문자열이 아닌 데이터는 리틀 엔디안 방식으로 저장된다.

- 비트 연산

비트 단위로 수행하는 연산. |, &, ^, ~, >>, << 연산자가 있다. & 연산자를 이용해서 비트 마스킹을 할 수 있고 & 연산과 시프트 연산을 활용하여 특정 위치의 비트를 가져올 수 있다.

XOR 연산 결과가 0인지 확인하면 두 값이 같은지 비교할 수 있다. XOR을 2번 하면 원래의 값과 동일해진다는 특성을 이용해 암호화, 복호화에 이용할 수 있다.

- 인코딩, 디코딩

인코딩은 데이터를 특정한 형식으로 변환하는 것을 말한다. 인코딩 방식으로는 아스키 코드와 유니 코드, url 인코딩과 base64 인코딩 등이 있다.

인코딩된 데이터는 디코딩하여 원래의 값을 구할 수 있다.

- 운영체제

사용자 및 응용 프로그램과 하드웨어 사이에서 중재자 역할을 하는 소프트웨어를 말한다. 하드웨어를 관리하고 사용자에게 편리함을 제공한다. Windows, UNIX, Linux, macOS, iOS 등이 있다.

운영체제는 커널과 셸로 나눌 수 있다. 커널은 하드웨어 관리를 실제로 수행하고, 소프트웨어와 하드웨어 간의 커뮤니케이션을 관리하며 시스템이 부팅될 때 메모리에 올라가서 꺼질 때까지 실행된다. 셸은 사용자와 운영체제의 커널 사이에서 사용자가 운영체제에 명령을 내릴 수 있도록 인터페이스 역할을 한다.

64se64

- 문제 설명

"Welcome!"을 출력하는 html 페이지입니다.

소스 코드를 확인하여 문제를 풀고 플래그를 획득하세요.

- 문제 풀이

웹페이지를 연 후 문제 설명대로 개발자 도구를 열어 소스 코드를 확인했다. base64로 인코딩 된 것으로 추정되는 값이 있어 디코딩 사이트를 이용하니 파이썬 코드가 나왔다. 파이썬 코드를 실행하니 바로 Flag를 얻을 수 있었다.

컴퓨터 아키텍처 기초

- 폰 노이만 구조

컴퓨터는 연산과 제어를 위해 중앙처리장치(CPU)를, 저장을 위해 메모리를 사용한다. 장치간에 데이터나 제어 신호를 교환할 수 있도록 하는 버스라는 통로를 이용한다.

CPU는 프로그램의 연산을 처리하고 시스템을 관리하는 컴퓨터의 두뇌다. 프로세스의 코드를 불러오고, 실행하고, 결과를 저장하는 일련의 모든 과정이 CPU에서 일어난다. 산술논리장치, 제어장치, 레지스터 등으로 구성된다.

기억장치(메모리)는 컴퓨터가 동작하는데 필요한 여러 데이터를 저장하기 위해 사용된다. 주기억장치와 보조기억장치로 분류된다.

버스는 컴퓨터 부품과 부품 사이 또는 컴퓨터와 컴퓨터 사이에 신호를 전송하는 통로를 말한다. 데이터 버스, 주소 버스, 제어 버스 등이 있다.

- 명령어 집합 구조

CPU가 해석하는 명령어의 집합을 말한다. CISC, RISC로 나눌 수 있다.

CISC는 메모리 효율성에 초점을 맞춰 개발되었다. RISC는 명령어의 개수가 적고 길이가 짧으며 고정된 길이의 명령어만 사용한다. CPU가 빠르게 디코딩할 수 있고 전력 효율이 좋기 때문에 저전력 기기에 적합하다.

- 범용 레지스터

주로 데이터 연산을 위해 사용되지만 그 외의 용도로도 자유롭게 사용할 수 있는 레지스터이다. x64에는 rax, rbx, rcx, rdx, rsi, rdi, rsp, rbp, r8-r15가 있다.

- 세그먼트 레지스터

메모리를 효율적으로 관리하거나 가용 메모리 공간의 확장을 위해 사용됐으나, 현재는 주로 메모리 보호를 위해 사용되는 레지스터이다. x64에는 cs, ss, ds, es, fs,gs가 있다.

- 플래그 레지스터

CPU가 연산을 수행한 후 결과의 상태를 저장한다. 특정 연산 후 조건을 판단하는 데 사용하며 분기, 인터럽트 등에 활용

한다.

- 명령어 포인터 레지스터

CPU가 실행해야할 코드를 가리키는 레지스터를 말한다. x64에서는 rip가 있다.

Linux 메모리 레이아웃

- 세그먼트

적재되는 메모리 영역을 용도별로 구획한 것을 말한다. 프로세스가 실행될 때 각 프로세스는 독립된 메모리 공간을 할당 받는데 이 메모리 공간은 용도에 따라 여러 영역으로 나뉘고 각 영역은 특정한 목적을 갖는다. 메모리 영역을 나눠 관리 하면 각 영역을 용도에 맞게 권한을 부여할 수 있다는 장점이 있다. 권한은 읽기(r), 쓰기(w), 실행(x) 세 가지가 존재한다.

- 코드 세그먼트(텍스트 세그먼트)

실행 가능한 기계어 코드가 위치하는 영역으로 읽기 권한과 실행 권한이 주어진다.

- 데이터 세그먼트

전역 변수 및 전역 상수가 위치한다. 쓰기가 가능한 data 세그먼트와 값이 변하면 안되는 데이터들이 위치하는 read-only data 세그먼트로 분류된다.

- BSS 세그먼트

컴파일 시점에 값이 정해지지 않은 전역 변수가 위치한다. 프로그램이 시작될 때 모두 0으로 값이 초기화된다.

- 힙 세그먼트

스택 세그먼트와 반대 방향으로 자란다. malloc(), calloc() 등을 호출해서 할당받는 메모리가 위치하게 되며 읽기와 쓰기 권한이 부여된다.

- 스택 세그먼트

함수의 매개변수나 지역 변수같은 임시 변수들이 저장된다. 스택이 확장될 때 기존 주소보다 낮은 주소로 확장된다. 읽기와 쓰기 권한이 부여된다.

x86 어셈블리

- x86-64 어셈블리어

명령어와 피연산자로 구성된다. 명령어에는 데이터 이동, 산술 연산, 논리 연산, 비교 등 다양한 명령어가 있다. 피연산자에는 상수, 레지스터, 메모리 3가지 종류가 올 수 있다.

- 산술 연산

add(덧셈), sub(뺄셈), mul, imul(곱셈), div, idiv(나눗셈)

- 논리 연산

and, or, xor, not

- 증감 연산

레지스터나 메모리에 저장된 값을 1씩 증가/감소시킨다. inc(1 증가), dec(1 감소)

- 데이터 저장과 이동

mov(데이터를 복사/이동), lea(유효 주소 저장)

- 비교

cmp 명령어의 결과에 따라 Zero Flag, Sign Flag, Overflow Flag가 설정된다.

- 분기

프로그램의 실행 흐름을 바꾸는 명령어로 명령어 포인터가 변경된다.

jmp(아무 조건 없이 무조건 지정된 주소로 이동), je/jz(ZF가 1이면 점프), jg/jge(두 피연산자 중 왼쪽 피연산자가 더 클 때, 더 크거나 같을 때 점프), jl/jle(두 피연산자 중 왼쪽 피연산자가 더 작을 때, 더 작거나 같을 때 점프), jne/jnz(두 피연산자가 다르면 점프)

- 반복문

카운터 기반 반복문, 조건 기반 반복문, 후조건 검사 반복문이 있다.

- 함수

어셈블리어에서 프로그램이 처리해야 할 명령어들을 한 덩어리로 모아 놓은 코드 블록을 말한다. call 명령어로 함수를 호출하고 ret 명령어로 다시 돌아간다.

스택 관련 명령어: push, pop

- 어셈블리어에서 함수 선언

함수 시작 위치를 나타낼 라벨을 정의한 후 스택 프레임 구성한다. 함수 내부에서 실제 동작을 구현한다. 함수 에필로그로 스택 프레임을 해제하고, ret 명령어로 종료한다.

- 시스템 콜

커널 모드: 운영체제가 전체 시스템을 제어하기 위해 시스템 소프트웨어에 부여하는 권한. 모든 메모리 영역에 접근할 수 있고, 하드웨어에 직접적으로 접근할 수 있다.

사용자 모드: 운영체제가 사용자에게 부여하는 권한으로, 접근할 수 있는 메모리 영역과 권한이 매우 한정되어 있고, 하드웨어에 직접적으로 접근할 수 없다.

시스템 콜: 사용자 모드에서 커널 모드의 시스템 소프트웨어에게 어떠한 동작을 요청하기 위해 사용한다.

함수 호출 규약

- x86 호출 규약

현대에는 대개 레지스터와 스택을 함께 사용한다.

cdecl: 리눅스의 gcc가 x86 바이너리를 컴파일할 때 일반적으로 사용하는 호출 규약이다. 레지스터 없이 스택만으로 인자를 전달한다.

stdcall: 윈도우 API에서 기본적으로 사용되는 함수 호출 규약이다. 스택 정리를 호출자가 아닌 피호출자가 한다.

fastcall: 함수 호출 시 일부 인자를 레지스터로 전달하여 속도를 향상시키기 위해 도입된 규약이다. 처음 두 인자를 각각 ecx, edx에 전달 후 나머지 인자는 마지막부터 스택으로 전달한다.

thiscall: C++ 클래스 멤버 함수를 위한 함수 호출 규약

- x86-64호출 규약: SYSV

리눅스는 SYSTEM V(SYSV) Application Binary Interface를 기반으로 만들어졌다. 6개의 인자를 RDI, RSI, RDX, RCX, R8, R9에 순서대로 저장하여 전달하며 호출자가 스택을 정리하고 함수의 반환값은 RAX로 전달한다.

